

Практическое занятие № 3

«Динамический массив и связный список»

Цель. Научиться реализовывать динамический массив и односвязный список, проверять их инварианты и осознанно выбирать структуру по профилю операций, фактическому времени и памяти.

Постановка задачи. Реализовать минимальные версии двух линейных структур, проверить одинаковый набор операций, провести контролируемый эксперимент на последовательности измерений и объяснить различия между асимптотической моделью и наблюдаемым результатом.

1.1. Планируемые результаты

- описывать логическое и физическое представление динамического массива и связного списка;
- формулировать инварианты размера, ёмкости, связности и хвостового указателя;
- реализовывать доступ, добавление, вставку, удаление и обход без смещения алгоритма с вводом-выводом;
- получать худшие и амортизированные оценки времени, разделять рабочую память и память структуры;
- строить воспроизводимую серию измерений и интерпретировать влияние локальности данных и накладных расходов объектов Python;
- обосновывать выбор структуры для данных физического эксперимента.

Уровень	Деятельность	Проверяемый артефакт
Базовый	Корректные операции и тесты границ	Код и протокол тестов
Средний	Инварианты и асимптотические оценки	Обоснование $T(n)$, $M(n)$
Продвинутый	Сравнительный эксперимент и выбор структуры	CSV, таблица, аргументированный вывод
ПК-4 / ПК-4.4	Программная реализация и вычислительный эксперимент	Отчёт, исходный код, данные

2. Инструменты и технологии

- Python 3.11 или новее; dataclasses или обычные классы, typing, time, statistics, tracemalloc, csv и gc;
- pytest допускается, но контрольные тесты должны запускаться и стандартным интерпретатором;
- редактор или среда разработки: VSCode, Visual Studio Code, PyCharm, JupyterLab либо командная строка;
- CSV для сырых измерений; DOCX или PDF для отчёта; UTF-8 для исходных файлов;
- Git рекомендуется для фиксации этапов, но не является обязательным.

Ограничение безопасности. Начинайте эксперимент с малых n . Не формируйте вход, способный вызвать подкачку или зависание общего учебного компьютера. Измеряйте время и память отдельно: tracemalloc заметно меняет время выполнения.

3. Краткие теоретические сведения

3.1. Динамический массив

Динамический массив хранит элементы в непрерывном логическом диапазоне и поддерживает ёмкость capacity не меньше фактического размера size . При переполнении выделяется увеличенный буфер, элементы копируются, после чего старый буфер освобождается. Доступ по индексу имеет $\Theta(1)$, вставка в середину – $\Theta(n)$, а добавление в конец при геометрическом росте имеет амортизированную стоимость $\Theta(1)$.

3.2. Односвязный список

Односвязный список состоит из узлов с данными и ссылкой next . Последовательный обход имеет $\Theta(n)$. Вставка после известного узла выполняется за $\Theta(1)$, но поиск позиции по индексу требует $\Theta(n)$. Для добавления в конец за $\Theta(1)$ нужен хвостовой указатель tail ; его корректность входит в инвариант структуры.

Операция	Динамический массив	Односвязный список
Доступ по индексу	$\Theta(1)$	$\Theta(n)$
Добавление в конец	амортизированно $\Theta(1)$, иногда $\Theta(n)$	$\Theta(1)$ при наличии tail
Вставка в начало	$\Theta(n)$	$\Theta(1)$
Удаление по индексу	$\Theta(n)$ из-за сдвига	$\Theta(n)$ из-за поиска
Обход	$\Theta(n)$, хорошая локальность	$\Theta(n)$, переходы по ссылкам
Накладная память	резерв незаполненной ёмкости	ссылка next и объект каждого узла

3.3. Инварианты

- для массива: $0 \leq \text{size} \leq \text{capacity}$; элементы с индексами $0 \dots \text{size}-1$ определены и идут в правильном порядке;
- для списка: head is None тогда и только тогда, когда $\text{size} = 0$; цепочка от head конечна и содержит ровно size узлов;
- если $\text{size} > 0$, tail указывает на последний достижимый узел и tail.next is None ;
- после каждой публичной операции все инварианты восстановлены.

4. Пошаговый план выполнения

1. Зафиксируйте контракт каждой операции: допустимые индексы, результат, исключения и изменение состояния.
2. Опишите внутреннее представление обеих структур и сформулируйте проверяемые инварианты.
3. Реализуйте динамический массив с операциями append , get , insert и pop ; рост ёмкости выполняйте геометрически.
4. Реализуйте односвязный список с head , tail и size ; добавьте append , prepend , get и pop_front .
5. До измерений выполните тесты пустой структуры, одного элемента, границ индекса, повторяющихся значений и длинной серии операций.
6. Определите профили нагрузки: последовательное добавление, вставка в начало и последовательное чтение.

7. Для каждого n выполните прогрев и не менее семи повторов; сохраните сырые времена и медиану.
8. Память измерьте отдельно; не включайте генерацию исходных данных в измеряемый участок.
9. Сопоставьте результаты с Θ -оценками и объясните влияние локальности, аллокаций и реализации Python.
10. Оформите отчёт, приложите код и CSV; раскройте использование внешнего кода и генеративного ИИ.

5. Демо-пример: журнал измерений

5.1. Условие

Поток датчика поступает по одному значению. Требуется хранить его в динамическом массиве и в односвязном списке, затем сравнить последовательное добавление, чтение всех значений и вставку калибровочной отметки в начало. Для демо используется компактная учебная реализация; встроенный list не заменяет собственный DynamicArray.

5.2. Реализация

```
from dataclasses import dataclass

class DynamicArray:
    def __init__(self):
        self._data = [None]
        self.size = 0

    @property
    def capacity(self):
        return len(self._data)

    def _resize(self, new_capacity):
        new_data = [None] * new_capacity
        for i in range(self.size):
            new_data[i] = self._data[i]
        self._data = new_data

    def append(self, value):
        if self.size == self.capacity:
            self._resize(max(1, 2 * self.capacity))
        self._data[self.size] = value
        self.size += 1

    def get(self, index):
        if not 0 <= index < self.size:
            raise IndexError(index)
        return self._data[index]

    def insert_front(self, value):
        if self.size == self.capacity:
            self._resize(max(1, 2 * self.capacity))
        for i in range(self.size, 0, -1):
            self._data[i] = self._data[i - 1]
        self._data[0] = value
        self.size += 1

@dataclass(slots=True)
class Node:
    value: float
    next: "Node | None" = None
```

```

class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def append(self, value):
        node = Node(value)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def prepend(self, value):
        node = Node(value, self.head)
        self.head = node
        if self.tail is None:
            self.tail = node
        self.size += 1

    def values(self):
        current = self.head
        while current is not None:
            yield current.value
            current = current.next
    
```

5.3. Контроль корректности

```

a = DynamicArray()
for x in (1.2, -0.5, 3.0):
    a.append(x)
a.insert_front(0.0)
assert [a.get(i) for i in range(a.size)] == [0.0, 1.2, -0.5, 3.0]
assert a.size <= a.capacity

s = SinglyLinkedList()
for x in (1.2, -0.5, 3.0):
    s.append(x)
s.prepend(0.0)
assert list(s.values()) == [0.0, 1.2, -0.5, 3.0]
assert s.size == 4 and s.tail.next is None
    
```

5.4. План эксперимента

Фактор	Решение
Размеры	$n = 1\,000, 2\,000, 4\,000, 8\,000, 16\,000, 32\,000$; уменьшить при долгом запуске
Повторы	1 прогревочный и не менее 7 измеряемых; итог – медиана
Операции	append n раз; prepend/insert_front для отдельного диапазона; полный обход
Данные	детерминированная последовательность $((37i \bmod 101) - 50) / 10$
Время	perf_counter_ns; создание входа вне таймера
Память	tracemalloc отдельным запуском; фиксировать пик в КиБ

Ожидаемая интерпретация. Обе структуры показывают линейное суммарное время п добавлений в конец, но динамический массив обычно быстрее при обходе. Серия вставок в начало имеет квадратичное суммарное время для массива и линейное – для списка. Это гипотеза, которую данные могут поддержать или опровергнуть на выбранном диапазоне, но эксперимент не заменяет доказательство сложности.

6. Индивидуальные задания для домашней работы

Для своего варианта реализуйте обе структуры или указанную модификацию, проверьте инварианты, подготовьте не менее пяти тестов и сравните минимум два профиля операций. Демо-пример не считается вариантом ДЗ.

№	Вариант	Условие
1	Буфер температуры	Хранить измерения, поддерживать append, чтение по индексу и удаление первых k значений; сравнить массив и список.
2	Редактор траектории	Поддерживать вставку точки после заданной позиции и последовательный экспорт; исследовать случай вставок в середину.
3	Очередь калибровок	Часто добавлять запись в начало и просматривать весь журнал; сравнить rprepend и полный обход.
4	Разреженная фильтрация	Удалять все значения вне диапазона $[a, b]$ с сохранением порядка; сравнить перестроение массива и переназначение ссылок.
5	Пакетный приём	Добавлять блоки разной длины; реализовать extend и оценить число перераспределений при коэффициентах роста 1.5 и 2.
6	Кольцевой маршрут	Хранить точки маршрута, вставлять контрольную точку перед первой ошибочной; сравнить стоимость поиска и вставки.
7	Дедупликация подряд	Удалить соседние повторы из серии измерений; обеспечить работу in-place для массива и однопроходную обработку списка.
8	Разбиение по порогу	Стабильно разделить значения на $< \tau$ и $\geq \tau$; сравнить дополнительную память и число проходов.
9	Слияние серий	Слить две упорядоченные серии в одну; для списка переиспользовать узлы, для массива создать результирующий буфер.
10	Контроль ёмкости	Реализовать shrink-to-fit и уменьшение ёмкости при малом заполнении; исследовать риск частых перераспределений.

7. Требования к отчёту

- титульная страница, номер варианта, цель и точная постановка задачи;
- контракты операций и инварианты обеих структур;
- псевдокод ключевых операций и оценки худшего/амортизированного времени;
- исходный код без смешения алгоритмов с консольным вводом-выводом;
- не менее пяти функциональных тестов, включая пустую структуру и границы индекса;

- таблица сырых времён, медиан и отдельная серия пиков памяти;
- сравнение теории и эксперимента, не менее трёх ограничений измерений;
- версия Python, ОС/архитектура, команда запуска и раскрытие внешнего кода или генеративного ИИ.

Компонент	Баллы	Критерий полного выполнения
Контракты и инварианты	20	Однозначные границы; инварианты проверяются после операций
Реализация	25	Обе структуры корректны; логика отделена от ввода-вывода
Тесты	15	Не менее пяти тестов, включая границы и длинную последовательность
Анализ сложности	15	Обоснованы худшие и амортизированные оценки времени и памяти
Эксперимент	15	Повторы, медиана, сырые данные и отдельное измерение памяти
Вывод и воспроизводимость	10	Выбор структуры обоснован; среда и ограничения указаны

8. Вопросы для самоконтроля

1. Чем логический размер динамического массива отличается от его ёмкости?
2. Почему единичное расширение буфера на каждый `append` приводит к квадратичному времени?
3. Как геометрический рост даёт амортизированную стоимость $\theta(1)$?
4. Какие условия составляют инвариант массива?
5. Какие условия составляют инвариант списка с `head` и `tail`?
6. Почему доступ по индексу в односвязном списке имеет $\theta(n)$?
7. Когда вставка в список действительно выполняется за $\theta(1)$?
8. Почему список может проигрывать массиву при последовательном обходе, хотя оба имеют $\theta(n)$?
9. Как изменится структура, если добавить ссылку `prev`?
10. Какая ошибка возникает, если при удалении последнего узла не обновить `tail`?
11. Почему память Python-объектов нельзя оценивать только размером полезного значения?
12. Какие входы проверяют границы индекса?
13. Почему прогрев и медиана повышают устойчивость временного эксперимента?
14. Почему измерение `tracemalloc` следует отделять от измерения времени?
15. Как по профилю операций выбрать между массивом и списком?
16. Почему эксперимент не доказывает асимптотическую оценку?

9. Источники

1. Актуальная РПД дисциплины «Алгоритмы и структуры данных» для направления 03.03.01, 2026/2027 учебный год.
2. Белов Владимир Викторович, Чистякова Валентина Ивановна, Алгоритмы и структуры данных: Учебник. – Москва: ООО "КУРС", – 2023.

3. Вирт, Н., Ткачев, Ф.В. Алгоритмы и структуры данных: Учебник. – Саратов: Профобразование, 2024.
4. Python Software Foundation. Python 3 Documentation: Data Structures, time, statistics, tracemalloc. <https://docs.python.org/3/> (дата обращения: 21.09.2026).

10. Лист самопроверки перед сдачей

- ☐ Номер варианта и постановка задачи указаны.
- ☐ Инварианты size, capacity, head и tail сформулированы и проверены.
- ☐ Все операции имеют контракт и тесты границ.
- ☐ Худшие и амортизированные оценки обоснованы.
- ☐ Сырые измерения сохранены, медиана посчитана, память измерена отдельно.
- ☐ Вывод связан с профилем операций и ограничениями эксперимента.
- ☐ Указаны среда, команда запуска и использование внешних средств.